

SAIL: Test-Time Scaling for In-Context Imitation Learning with VLM

Makoto Sato¹, Yusuke Iwasawa¹, Yujin Tang², and So Kuroki²

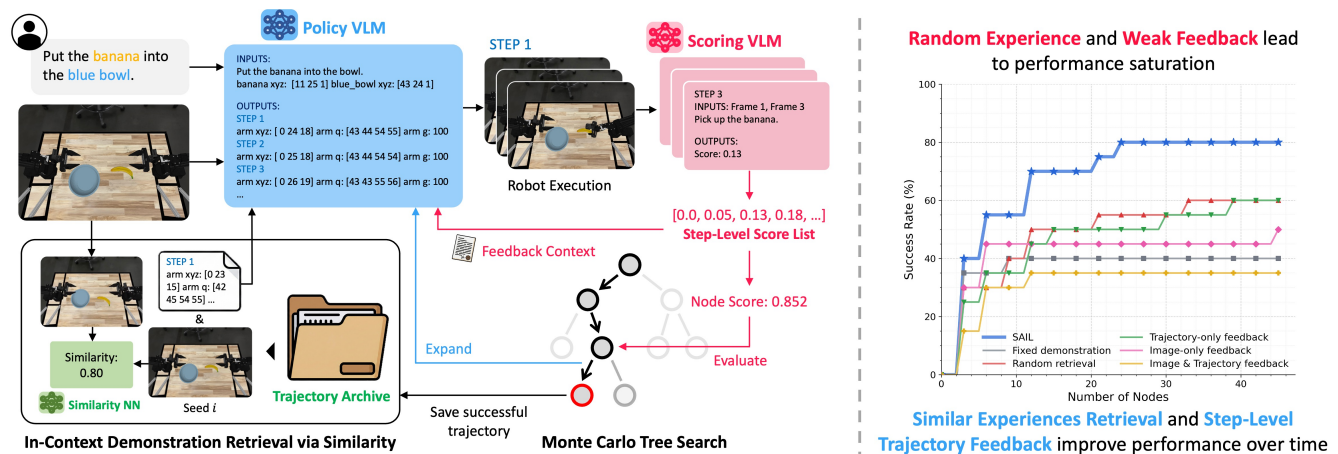


Fig. 1: **Left:** We perform test-time scaling with MCTS over VLM-proposed trajectories, using similarity-based retrieval from an archive of successful rollouts and step-level feedback from a VLM evaluator. **Right:** Success improves with more expanded MCTS nodes (test-time compute); random retrieval and weak feedback saturate early, whereas our retrieval and step-level feedback keep yielding gains.

Abstract—In-context imitation learning allows robots to acquire skills from demonstrations, yet one-shot trajectory generation remains fragile under environmental variation. We propose SAIL, a framework that reframes robot imitation as an iterative refinement problem capable of scaling with test-time compute. SAIL utilizes Monte Carlo Tree Search, where each node is a complete trajectory and edges correspond to trajectory refinements. The process is guided by three core components: an automated archive of successful trajectories for contextually relevant retrieval, a vision language model-based scoring mechanism for trajectory evaluation, and a step-level feedback that provides trajectory-aligned scores for iterative refinement. Experiments across six diverse manipulation tasks in simulation and real-world validation clearly demonstrate that increasing test-time compute consistently improves success rates, achieving up to 95% on complex tasks. Our results suggest that trajectory-level test-time scaling is a robust path toward more generalizable robotic agents.

I. INTRODUCTION

In-context learning has emerged as a compelling paradigm for robots to acquire complex skills directly from a few visual hints and trajectory demonstrations [1], [2], [3]. However, a fundamental bottleneck remains because a single realized trajectory often fails to generalize when the robot encounters new initial conditions. While current vision language models (VLMs) can generate trajectories [4], [5], they typically treat this as a one-shot prediction task. This limitation means that the success of the system is capped by the initial guess of the model, which cannot adjust for environmental variations during the inference pass.

In this work, we propose that robot imitation should instead be framed as an iterative search problem where performance can be improved by scaling test-time compute. By treating full trajectories as the objects of search, we enable the system to explore potential solutions in a trial-matched simulator before execution. This approach shifts the focus from static imitation to a dynamic paradigm where a robot can effectively think longer to resolve the ambiguities of a novel task.

The reliance on foundation models’ one-shot inference is problematic because the high dimensional space of robot trajectories is extremely sensitive to small errors in initial state estimation or object localization. Even a slight misalignment in the predicted end effector pose can lead to cascading failure during execution which the model cannot correct without an external loop for evaluation. Recent efforts have attempted to mitigate this fragility by either distilling procedural reasoning into more robust visual representations [6], [7], or employing retrieval mechanisms to provide the model with richer context from successful historical demonstrations [8]. However, these methods typically focus on improving the quality of a single prediction rather than enabling the system to systematically explore and refine the continuous motion space itself. While some frameworks incorporate external validators to guide corrections [9], [10], they often remain limited to symbolic planning and do not address the fundamental need for test-time scaling at the level of full robot trajectories.

To address these challenges we introduce SAIL (Scaling In-context Imitation Learning), a novel framework that enables in-context imitation to scale through test-time compute

*This work was done while Makoto Sato was an intern at Sakana AI.

¹The University of Tokyo, Tokyo, Japan.

²Sakana AI, Tokyo, Japan.

(see Figure 1 left). Rather than relying on a VLM’s single prediction, SAIL treats trajectory generation as a refinement problem, and introduces three key components to guide the refinement process toward success. First, we formulate trajectory generation in a Monte Carlo Tree Search (MCTS) setting where each node represents a complete robot trajectory. This enables the system to systematically explore the continuous motion space rather than depending on a single one-shot plan. Second, we maintain an automated archive of successful trajectories which are retrieved to provide contextually relevant demonstrations for new environmental configurations. This retrieval mechanism allows the system to bootstrap its search by leveraging past experiences in visually similar scenes. Finally, we propose a step-level feedback that uses trajectory-aligned scores to annotate trajectories. This allows the generator to identify specific points of failure and iteratively refine its motion based on granular critiques.

We evaluate SAIL across six diverse manipulation tasks in simulation and validate the full pipeline on a real robot system. Our results demonstrate a clear scaling property where increasing the test-time compute budget consistently leads to higher success rates across all tasks (see Figure 1 right). Notably, the system achieves a 95% success rate on the HandOverBanana task and successfully transfers to the physical world with a high success rate on a block moving task using a trial matched digital twin. These findings suggest that scaling search at the trajectory level is a viable path toward more robust and generalizable robotic agents.

Our contributions can be summarized as follows: (1) We reformulate in-context robot imitation as a trajectory-level test-time refinement problem, enabling performance to scale beyond one-shot trajectory prediction. (2) To realize this formulation, we develop SAIL that combines trajectory-level MCTS, retrieval-augmented demonstrations, and step-level evaluation from a VLM for iterative refinement. (3) We demonstrate through extensive experiments in simulation and the real world that increasing test-time compute leads to consistent improvements in task success rates.

II. RELATED WORK

A. Trajectory Generation Using LLMs and VLMs

Recent work has demonstrated that large language models (LLMs) and VLMs can be used not only for high-level planning but also for direct generation of executable robot trajectories. Early approaches show that an off-the-shelf LLM can generate a complete trajectory in a single forward pass, producing dense pose sequences that are directly executable in simulation or on real robots [11]. This direct but unconstrained generation is often brittle, leading to subsequent work that introduces more structured representations to improve robustness, such as tokenizing observations and actions or converting demonstrations into structured textual templates for in-context trajectory or action prediction [2], [12]. Related efforts further emphasize structural conditioning by moving from dense trajectory generation to waypoint-level representations explicitly constrained by object kinematics [13]. In parallel, trajectory editing methods address

similar robustness issues by modifying an existing trajectory using language-derived waypoint updates or geometric constraints rather than generating a new trajectory from scratch [14], [15]. Overall, these studies establish that direct trajectory generation with foundation models is feasible and can be stabilized through representation design and output-space constraints; however, trajectory generation is still typically performed as a one-shot or lightly iterative process for a single environment instance, and systematic test-time scaling across many initial conditions remains largely unexplored, which this work addresses.

B. Planning and Test-Time Scaling

Recent work in robot planning increasingly combines LLMs with external evaluation modules and search processes, rather than treating LLMs as standalone plan generators. In this setting, planning is formulated as an iterative process in which generated plans are examined for feasibility and revised based on detected failures or inconsistencies, instead of being finalized in a single forward pass [16], [17]. For long-horizon tasks, verifying plans through constraint satisfaction or motion planning and incorporating the resulting feedback into subsequent generations has been shown to substantially improve executability [16]. Beyond natural language plans, several studies propose generating plans in structured and verifiable intermediate representations, such as symbolic code or constrained programs, enabling external validators to guide corrections and framing planning as an optimization process [18], [19]. Retrieval-based designs further extend these approaches by reusing past successful experiences at inference time, allowing adaptation to new environments through in-context retrieval [20]. Building on these foundations, several works explore test-time scaling by incorporating external scoring functions or search mechanisms, such as candidate selection based on external evaluations [21], systematic expansion of candidate solutions via MCTS guided by LLM outputs [22], and iterative replanning supported by structured environment representations [23]. While these approaches improve robustness and scalability of planning, they primarily focus on task-level decisions or planning structures rather than continuous motion generation. In contrast, our work targets test-time scaling at the level of continuous robot trajectory generation itself, treating full trajectories as the objects of search and evaluation.

C. Scoring Using VLMs

Research on evaluating robot executions as videos has expanded with the development of VLMs, moving from success detection toward continuous scoring that captures the execution process. Early work used VLMs to judge success or failure by checking the consistency between post-execution observations and natural language task descriptions, showing that robot execution videos can serve as direct evaluation targets [9], [10]. Subsequent studies considered video sequences as input and estimated task progress or value along the temporal dimension. For instance, Generative Value Learners (GVL) demonstrated that presenting frame

sequences in context and reasoning over their temporal order can produce scores related to progress and value during execution [24]. Other VLM-based reward and evaluation models explicitly focus on manipulation processes and stage structures, including work on stage recognition for long-horizon manipulation, systematic studies of vision-language reward models for robot execution videos, and process-level evaluation of manipulation trajectories [25], [26], [27]. In addition, several studies use VLMs to detect planning or execution errors from execution videos and to classify failure types, treating video evaluation as a diagnostic signal rather than a binary outcome [28], [29]. Across these lines of work, an important question is not only how to assign a single scalar score to an entire video, but also how to extract and use information along the execution timeline at an appropriate temporal resolution. In this context, ROVER addresses long robot execution videos by explicitly modeling temporal structure, recursively segmenting videos into subtask-aligned intervals and assigning scores at the timestep level [30]. Following this trend, we use the VLM-based scorer to obtain both a trajectory-level score for search and timestep-level scores for feedback during robot trajectory generation.

III. BACKGROUND

A. Problem Formulation

We consider a target manipulation task (e.g., handing over a banana or a pen) under multiple initial environmental conditions, indexed by a random seed. Let $\omega \in \Omega$ denote a seed, and let $\mathbf{I}_\omega$ be the initial RGB observation for seed ω . For a small subset of seeds $\Omega' \subset \Omega$, we assume access to one successful trajectory demonstration per seed, denoted by $\tau_{\omega \in \Omega'}^{(v)}$. Our goal is to generate a successful robot trajectory $\tau_{\omega \in \Omega \setminus \Omega'}$ for an unseen seed by leveraging these initial demonstrations.

The trajectory space is defined over the robot end-effector state $\mathbf{e}_\omega$, represented as a tuple of Cartesian position $\mathbf{p}_\omega \in \mathcal{R}^3$, orientation quaternion $\mathbf{q}_\omega \in \mathcal{R}^4$, and gripper status $g_\omega \in \{\text{OPEN}, \text{CLOSE}\}$. A complete trajectory for seed ω is a sequence $\tau_\omega = (\mathbf{e}_\omega^{(1)}, \dots, \mathbf{e}_\omega^{(T)})$.

To enable in-context imitation beyond one-shot prediction, we formulate inference as an iterative refinement process that runs independently per seed. For a fixed seed ω , at refinement step v^1 , a policy VLM proposes a *complete* trajectory $\tau_\omega^{(v)}$ conditioned on (1) the initial state $\mathbf{s}_\omega$, (2) a set of in-context reference demonstrations $\{\tau_{\omega_k}^{(v)}\}_{k=1}^K$ inserted into the prompt at step v , and (3) feedback derived from previously executed trajectories along the current search path. The policy VLM then generates $\tau_\omega^{(v)}$, which is then executed and evaluated to drive subsequent refinements, until a successful trajectory τ_ω^* is obtained.

To construct the initial state representation $\mathbf{s}_\omega$, we extract task-relevant objects' spatial information from the initial RGB observation $\mathbf{I}_\omega$. Concretely, a detection VLM predicts 2D keypoints for task-relevant objects, which are lifted to

3D world coordinates using calibrated camera parameters. We form $\mathbf{s}_\omega$ by concatenating the 3D keypoints with the initial robot state $\mathbf{e}_\omega$.

B. Monte Carlo Tree Search

We formulate trajectory refinement as a MCTS problem, where each node corresponds to a complete trajectory and edges correspond to refinement operations that modify a previous trajectory. For each evaluation seed $\omega \in \Omega$, we build an independent search tree. Each node corresponds to a complete trajectory proposal $\tau_\omega^{(v)}$, generated by the policy VLM conditioned on the initial state $\mathbf{s}_\omega$ and a node-specific in-context demonstration set. We execute each proposal in simulation and obtain an evaluation signal.

We use a standard MCTS loop consisting of *selection*, *expansion*, *evaluation*, and *backup*. For a node n , let $N(n)$ denote its visit count and let $\bar{R}(n)$ denote its empirical mean value. During selection, starting from the root, we repeatedly choose a child $c \in \text{Ch}(n)$ that maximizes the Upper Confidence Bound (UCB) score, which balances exploitation and exploration. In this work, we adopt a prior-weighted UCB (PUCB [31]) variant that incorporates a node-specific prior to further enhance exploration toward promising branches:

$$c^* = \arg \max_{c \in \text{Ch}(n)} \left(\bar{R}(c) + c_{\text{pucb}} * P(c) \sqrt{\frac{\ln N(n)}{N(c)}} \right), \quad (1)$$

where $c_{\text{pucb}} > 0$ is the exploration coefficient and $P(c)$ denotes a prior term proportional to the node's score. During expansion, we generate B child nodes from the selected leaf by sampling B refined trajectories from the policy VLM (we refer to B as the *branching factor*). Each child trajectory is executed and scored to obtain a scalar reward (Sec. IV-B), and these rewards are then backed up along the selected path to update $N(\cdot)$ and $\bar{R}(\cdot)$. We terminate when the test-time compute budget is exhausted (or early-stop once a successful trajectory is found) and return the best trajectory encountered.

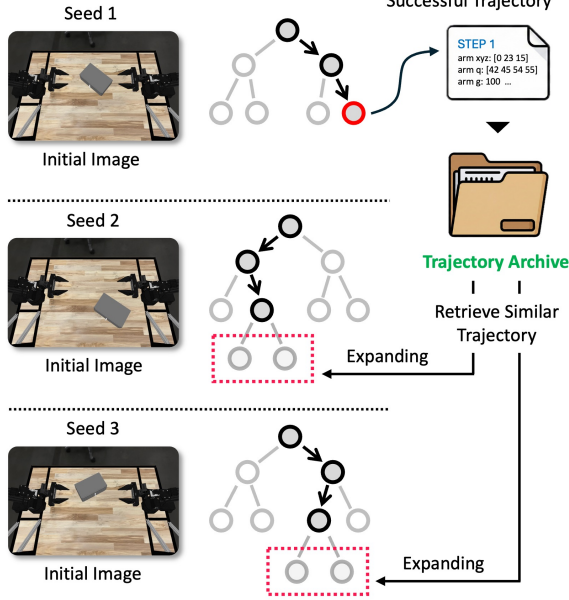
IV. METHODS

SAIL iteratively proposes, evaluates, and refines complete trajectories in a simulator. Figure 2 shows the method overview. At a high-level, the policy VLM receives a discretized numeric encoding of $\mathbf{s}_\omega$, together with the in-context reference demonstrations and feedback from previous attempts, and outputs a candidate trajectory $\tau_\omega^{(v)}$. An inverse kinematics (IK) controller executes the waypoints sequentially. Then, a scoring mechanism evaluates the resulting rollout and returns both a scalar node value and step-level signals, which are used to guide iterative refinement in the next step.

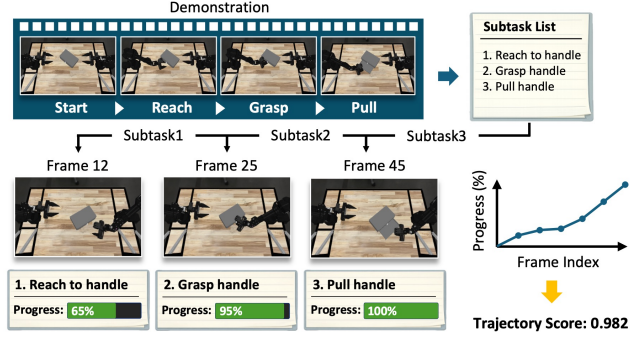
In terms of test-time scaling for trajectory refinement, SAIL consists of the following components that allows it to reuse successful trajectories across seeds, evaluate each candidate to guide node selection, and return dense feedback to refine subsequent proposals.

¹Please notice that $t \in [1, T]$ indicates the trajectory execution steps, and $v \in [1, V]$ is the test-time scaling refinement iteration.

1. Trajectory Archive for MCTS



2. VLM Scoring



3. Step-Level Dense Feedback

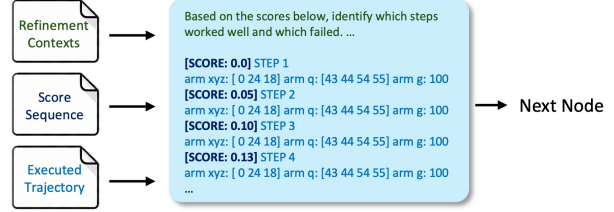


Fig. 2: **Method overview.** SAIL refines trajectories at test time via MCTS, where each node is a complete trajectory proposed by a policy VLM for a seed. (1) A shared *trajectory archive* stores successful rollouts across seeds and retrieves visually similar trajectories as in-context demonstrations. (2) Each proposal is executed and scored by a scoring VLM that (i) decomposes the task into ordered subtasks from one demo and (ii) estimates per-frame completion to yield a scalar node value. (3) The scoring VLM aligns the progress scores to waypoints to provide step-level feedback for the next refinement.

- 1) **Archive Retrieval for In-Context Demonstrations** (Sec. IV-A): We maintain a shared archive of successful trajectories across seeds and retrieve visually similar trajectories for each node.
- 2) **Trajectory Scoring for Node Evaluation** (Sec. IV-B): We execute a generated trajectory in a simulator and use a scoring VLM to compute a scalar reward as the node value.
- 3) **Step-Level Feedback for Trajectory Refinement** (Sec. IV-C): We compute a progress signal from uniformly sampled rollout video frames and align these scores to the corresponding waypoints to provide dense feedback for the next refinement.

A. Archive Retrieval for In-Context Demonstrations

In in-context imitation learning, performance depends strongly on how well the provided demonstrations match the test-time scene. We therefore maintain an *archive* of successful trajectories that grows as the system searches across seeds, enabling retrieval of scene-compatible references over time. We define the archive as a set of successful rollouts:

$$\mathcal{A} = \{(\mathbf{I}_i, \tau_i)\}_{i=1}^{|\mathcal{A}|}, \quad (2)$$

where $\mathbf{I}_i \in \mathcal{R}^{H \times W \times 3}$ is the initial RGB observation and τ_i is the corresponding successful trajectory. We initialize the archive with the provided demonstrations across seeds, i.e., $(\mathbf{I}_\omega, \tau_\omega) \in \mathcal{A}$ for each $\omega \in \Omega'$. During search, whenever we discover a successful trajectory for a seed ω , we append

$(\mathbf{I}_\omega, \tau_\omega^*)$ to $\mathcal{A}$. As a result, $\mathcal{A}$ contains successful trajectories originating from multiple seeds, and can be reused when searching for other seeds.

When expanding a node for a target seed ω with initial observation $\mathbf{I}_\omega$, we retrieve a small set of visually similar archive entries to form the in-context examples. We compute perceptual similarity using the LPIPS distance [32] and select the K nearest neighbors by minimizing the total distance:

$$\mathcal{S}^*(\mathbf{I}_\omega) = \arg \min_{\substack{S \subseteq \{1, \dots, |\mathcal{A}|\} \\ |S|=K}} \sum_{i \in S} \text{LPIPS}(\mathbf{I}_\omega, \mathbf{I}_i). \quad (3)$$

Let $\mathcal{S}^*(\mathbf{I}_\omega) = \{i_1, \dots, i_K\}$. We define $\tau_{\omega_k}^{(v)} := \tau_{i_k}$ for $k = 1, \dots, K$, and use $\{\tau_{\omega_k}^{(v)}\}_{k=1}^K$ as in-context references.

B. Trajectory Scoring for Node Evaluation

Designing task-specific reward functions across diverse manipulation tasks is labor-intensive and does not scale. We adopt a general VLM-based scoring mechanism that evaluates the executed rollout video and outputs a scalar reward for search.

A binary success signal is often too sparse for long-horizon tasks. Following the trend of assigning timestep-level scores to capture the execution process [30], we evaluate progress over an ordered sequence of intermediate subtasks (e.g., reach, grasp, lift, handover in a hand over pen task). To obtain these subtasks, we pre-generate a temporally ordered subtask list once per task using one available demonstration

τ_ω from a seed $\omega \in \Omega'$. Specifically, we provide the scoring VLM with the final goal specification and the rollout video of τ_ω , and prompt it to decompose the task into M ordered natural-language subtasks $\{\ell_m\}_{m=1}^M$.

Given $\{\ell_m\}_{m=1}^M$, we score a candidate trajectory for seed ω by executing $\tau_\omega^{(v)}$ in simulation and querying the VLM sequentially to estimate completion progress of the *current* subtask. For each subtask ℓ_m , we treat the first frame of that subtask as a start reference (0% completion) and evaluate subsequent frames in temporal order. We uniformly sample N rollout video frames along the temporal axis of the executed trajectory. At each evaluated rollout video frame $f \in [1, N]$, we query the VLM with the current subtask, the start reference frame, and the current frame, and parse its output into a completion percentage $\text{VLM}(\ell_m, \tau_\omega^{(v)}, f) \in [0, 100]$ (i.e., we prompt the VLM to directly output the completion percentage). When $\text{VLM}(\ell_m, \tau_\omega^{(v)}, f)$ reaches 100%, we mark the subtask complete and proceed to ℓ_{m+1} . If the rollout ends before completion, we stop and treat the trajectory as failing at that subtask.

We convert this subtask-level progress into a per-frame progress score $r_\omega^{(v)}(f) \in [0, 1]$:

$$r_\omega^{(v)}(f) = \frac{1}{M} \left((m_\omega^{(v)}(f) - 1) + \frac{\text{VLM}(\ell_m, \tau_\omega^{(v)}, f)}{100} \right), \quad (4)$$

where $m_\omega^{(v)}(f) \in \{1, \dots, M\}$ denotes the current subtask index at rollout video frame f for seed ω . We then compute a scalar reward by averaging progress across N rollout video frames and use it as the node value in MCTS.

C. Step-Level Feedback for Trajectory Refinement

To refine trajectories effectively, the policy VLM benefits from feedback that localizes where progress stalls. We provide step-level feedback by annotating the proposed trajectory with the progress scores produced by the scorer.

Given the sampled progress curve $\{r_\omega^{(v)}(f)\}_{f=1}^N$ from Sec. IV-B, we align each sampled score to its corresponding waypoint in the generated trajectory $\tau_\omega^{(v)} = (\mathbf{e}_\omega^{(1)}, \dots, \mathbf{e}_\omega^{(T)})$ based on the uniform sampling schedule. This produces an *annotated trajectory* in which selected waypoints are tagged with their completion scores. During the next expansion step for the same seed ω , we include this annotated trajectory in the prompt as dense feedback, together with the retrieved demonstration set $\{\tau_{\omega_k}^{(v)}\}_{k=1}^K$. The policy VLM is instructed to preserve high-scoring segments while modifying low-scoring segments, enabling targeted corrections during iterative refinement.

V. EXPERIMENTS

In this section, we experimentally evaluate our method in both simulation and the real world. Through our experiments, we aim to address the following research questions:

- Does increasing the test-time compute budget (MCTS nodes) improve task success rates? (Sec. V-B)

- How do automated archive retrieval and step-level feedback contribute to iterative trajectory refinement? (Sec. V-C)
- Can trajectories refined in simulation be successfully transferred to the real world? (Sec. V-D)

A. Tasks and Experimental Setup

In simulation, we consider six manipulation tasks in the ALOHA manipulation simulator [33]: *HandOverBanana* (HOB), *HandOverPen* (HOP), *BowlOnRack* (BOR), *DrawerOpen* (DO), *LaptopClose* (LC), and *MarkerRemoveLid* (MRL). Each task instance uses a random seed to determine the initial object configurations. For every task, we provide one or a few successful reference trajectory demonstrations generated from seed(s) distinct from the evaluation seeds. We evaluate 20 different seeds per task and determine success using the simulator’s ground-truth verification function. We use an overhead camera view and the `gemin-robotics-er-1.5-preview` model for all VLM-based tasks including 2D keypoint detection, trajectory generation, and trajectory scoring. During MCTS, we set branching factor $B = 3$. Unless otherwise specified, we retrieve $K = 1$ demonstrations for the automated archive retrieval. For trajectory scoring, we sample $N = 50$ frames at regular intervals from the rollout video. We define the task success rate as the percentage of the 20 evaluation seeds where our method generates a successful trajectory.

In Section V-D, we evaluate our pipeline on a real-world *BlockIntoBowl* task using a LeRobot SO-101 [34] arm. We conduct 6 randomized tabletop trials. For each trial, we select a trajectory refined in a trial-specific Real-to-Simulation (Real2Sim) environment. We then execute this trajectory on the physical robot via Simulation-to-Real (Sim2Real) transfer and report the success rate. We maintain the same hyperparameter settings used in the simulation experiments.

B. Scaling with Test-Time Compute

We study whether increasing the test-time compute budget improves task success. Table I summarizes our results.

Scaling Test-Time Compute: We scale the trajectory search up to a maximum budget of 45 nodes and compare it against a conventional single rollout generation. Overall, success improves consistently as we allocate more compute. The average success rate across all tasks increases from 25% with a single rollout to 73% with 45 MCTS nodes. This substantial gain demonstrates that iterative refinement through test-time compute effectively enhances trajectory generation. In particular, success rates rise across several tasks—for example, *DO* improves from 10% to 50%, *LC* from 15% to 70%, and *MRL* from 5% to 45%. Meanwhile, some tasks plateau early; *BOR* reaches 100% success by 6 nodes and remains at 100% across larger budgets.

Search Strategy Comparison: We further evaluate different search strategies under a fixed budget of 15 nodes. Breadth-first search expands 15 nodes in parallel without a feedback loop, whereas depth-first search expands one node at a time through 15 sequential refinement steps. Under this budget,

TABLE I: Success rates over 20 seeds under different test-time compute budgets measured by expanded MCTS nodes. Gray rows indicate the 15-node budget for fair comparison. Bold denotes the best result among the 15-node methods, and underline denotes the best overall.

Method	Nodes	Success Rate						Avg
		<i>HOB</i>	<i>HOP</i>	<i>BOR</i>	<i>DO</i>	<i>LC</i>	<i>MRL</i>	
Single rollout	1	0.40	0.40	0.40	0.10	0.15	0.05	0.25
Breadth-search	15	0.85	0.50	0.60	0.35	0.60	0.15	0.51
Depth-search	15	0.45	0.50	0.80	0.05	0.30	0.10	0.37
Ours	6	0.80	0.55	1.00	0.20	0.50	0.25	0.55
	15	0.90	0.70	1.00	0.40	0.50	0.40	0.65
	30	0.95	0.80	1.00	0.50	0.55	0.45	0.71
	45	0.95	0.80	1.00	0.50	0.70	0.45	0.73

our MCTS approach achieves an average success rate of 65%, compared to 51% for breadth-first search and 37% for depth-first search. While MCTS performs best on average, breadth-first search attains a higher success rate on the *LC* task. Closing the laptop requires fine positional adjustments to safely pass the gripper over the thin upper edge of the screen, and the stochastic exploration of breadth-first search handles this specific geometric requirement well. Overall, these results indicate that our method succeeds because it effectively combines wide trajectory exploration with vertical step-level feedback.

C. Effects of Retrieval Strategy and Feedback Modality

To assess the impact of our retrieval strategy and feedback mechanism, we conducted an ablation study with a fixed search budget of 15 nodes. Table II summarizes the results. **Retrieval Strategy:** To evaluate the impact of our archive and retrieval strategy on performance, we compare our approach with two baselines. The *fixed demonstration* baseline disables the archive and uses only the original in-context demonstrations. The *random retrieval* baseline selects K archive entries uniformly at random.

Table II shows that our method with similarity-based retrieval (Ours) improves the average success rate over both the fixed demonstration and random retrieval baselines when $K = 1$, and outperforms them on most tasks. In particular, our method achieves an average success rate of 65%, compared to 45% with a fixed demonstration and 50% with random retrieval. These results show that similarity-based retrieval yields stronger performance than both the fixed demonstration and random retrieval baselines under the same context size.

We further observe that increasing the number of in-context demonstrations alone does not reliably close this performance gap. When we increase the context size for fixed demonstrations from $K = 1$ to $K = 3$, the average success rate improves only modestly, from 45% to 49%. A similar trend holds for random retrieval, which increases from 50% to 53% on average. Both remain below similarity-based retrieval with only a single retrieved demonstration, which achieves 65% on average. This pattern is also reflected in individual tasks; for example, on *DO*, fixed demonstrations

improve from 5% to 10%, while similarity-based retrieval reaches 40% with $K = 1$. Overall, these results suggest that performance is more sensitive to the relevance of in-context demonstrations than to their sheer quantity, and that allocating context budget to more similar demonstrations is more effective than simply adding more demonstrations.

Feedback Modality: To evaluate our *Step-level feedback*, we compare this against three baselines: (i) *Trajectory-only*: provides only the textual action sequence of the previous attempt; (ii) *Image-only*: provides a sequence of rollout frames, subsampled to correspond to the trajectory steps; and (iii) *Image & Trajectory*: provides both the trajectory text and the corresponding rollout frames, but without the step-wise score annotations.

Step-level feedback (Ours) yields the strongest refinement performance across tasks. Compared to dense baselines that provide only raw history without explicit scores, our step-level feedback variant achieves a higher average success rate (65% vs. 45–48%). This improvement is also reflected in individual tasks; for example, on *LC*, step-level feedback reaches 50% success, while the dense baselines achieve 30–35%. These results indicate that visual or textual rollouts alone are not sufficient to reliably guide refinement; explicit step-level scores help the model localize where the trajectory first deviates from success and what needs to be corrected.

We also compare against sparse feedback that provides only a final score. While sparse feedback can help in some cases, it is consistently weaker than step-level feedback. For example, sparse feedback attains 15% on *DO* and 30% on *MRL*, whereas step-level feedback achieves 40% on both tasks. Overall, the ablation suggests that refinement benefits most from dense, score-aligned feedback. Because the step-wise annotations are produced by the same scoring function used to evaluate rollouts, the refinement signal is directly aligned with the objective, leading to more reliable improvements than unscored histories or sparse end-of-trajectory feedback.

We further scaled several ablation experiments up to 45 nodes, where SAIL clearly demonstrated superior performance (see details in Figure 1 right).

D. Real-World Validation

We validate our approach on a real-world *BlockIntoBowl* task using a LeRobot SO-101 arm. Figure 3 illustrates the experimental setup. The task is to place a target block into a red bowl. First, we use a Real2Sim pipeline to reconstruct the physical workspace in simulation. Next, we apply our search method within this simulation to generate a successful trajectory. We then execute this trajectory on the physical robot through Sim2Real transfer. We conduct six trials. For each trial, we manually randomize the initial positions of the block and the bowl on a tabletop and run the complete pipeline from perception to execution.

1) *Real2Sim Pipeline:* To reconstruct the physical workspace in simulation, our Real2Sim pipeline processes synchronized RGB-D observations from a fixed Intel RealSense D435i camera to estimate the 6-DoF poses of the

TABLE II: Ablation study with a fixed MCTS budget of 15 nodes (success rate). We compare retrieval strategies and feedback modalities for refinement.

Setting	Success Rate						
	HOB	HOP	BOR	DO	LC	MRL	Avg
Ours ($K=1$)	0.90	0.70	1.00	0.40	0.50	0.40	0.65
<i>Retrieval Strategy ($K=1$)</i>							
Fixed demonstration	0.75	0.40	0.85	0.05	0.50	0.15	0.45
Random retrieval	0.75	0.50	0.90	0.15	0.55	0.15	0.50
<i>Retrieval Strategy ($K=3$)</i>							
Fixed demonstration	0.80	0.35	0.95	0.10	0.45	0.30	0.49
Random retrieval	0.90	0.60	0.85	0.15	0.45	0.20	0.53
<i>Feedback Modality (Dense)</i>							
Trajectory-only	0.85	0.50	0.95	0.10	0.35	0.10	0.48
Image-only	0.75	0.45	0.95	0.10	0.35	0.10	0.45
Image & trajectory	0.80	0.35	0.95	0.15	0.30	0.20	0.46
<i>Feedback Modality (Sparse)</i>							
Final score	0.80	0.40	0.90	0.15	0.40	0.30	0.49

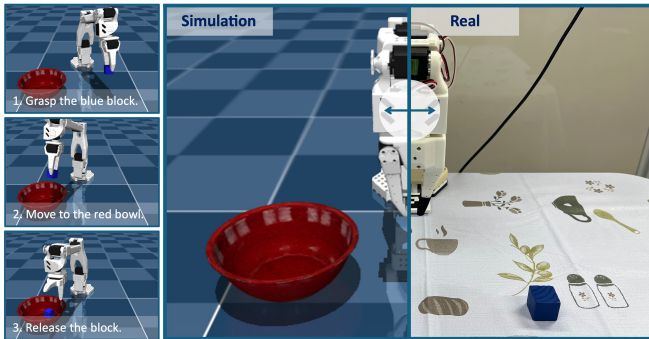


Fig. 3: Digital-twin experimental setup for the *BlockIntoBowl* task. The task consists of three sequential steps: (1) grasp the blue block, (2) move to the bowl, and (3) release the block into the bowl. The left panel shows the simulation environment, and the right panel shows the corresponding real-world setup.

target objects. First, we localize the target objects in an RGB frame using GroundingDINO [35] with the text prompts `blue_block` and `red_bowl`, and refine the detections into instance masks using SAM2 [36]. We then use the aligned depth data to lift each mask into a 3D point set. To map these points to the robot’s coordinate system, we apply offline-calibrated camera intrinsics and runtime extrinsics estimated from an ArUco marker fixed at a known location on the robot base (0.1 m along the X-axis). Using these parameters, we aggregate the masked depth points and fit a 3D bounding box to compute the 6-DoF object poses. Finally, we instantiate the scene-matched simulator by placing the corresponding simulation assets at these estimated poses and matching the simulated camera viewpoint to the real setup.

2) *Sim2Real: Trajectory and Policy Deployment*: After constructing the trial-specific digital twin, we generate a successful trajectory within the simulation and execute it on the real robot. We evaluate two methods for generating this trajectory: our MCTS-based search and a policy distilled from MCTS rollouts. We report success rates over six real-world trials for each method. We consider a trial successful if the robot places the target block inside the red bowl. The

TABLE III: Real-world *BlockIntoBowl* success rates over 6 trials.

Method	Success Rate
MCTS-based Trajectory Refinement	5 / 6
Sim2Real Policy Distillation	5 / 6

table III shows the results.

MCTS-based Trajectory Refinement: In this approach, we run our MCTS method directly within the simulation environment constructed via Real2Sim. We allocate a maximum budget of 15 search nodes. At each node, the policy VLM proposes candidate trajectories conditioned on the current scene observation and retrieved in-context demonstrations. We terminate the search early once we find a successful trajectory and execute it on the real robot. This method succeeded in 5 out of 6 real-world trials. This high success rate demonstrates that trajectories validated in the digital twin effectively transfer to the physical world. We attribute the single failure to the Sim2Real gap, specifically slight errors in RGB-D pose estimation and unmodeled real-world contact dynamics.

Sim2Real Policy Distillation: While the MCTS-based search achieves high performance, the iterative trajectory generation process is time-consuming. To address this limitation, we distill the MCTS-based search into a learned policy. First, we collect a dataset of 120 successful trajectories by randomizing initial object placements in simulation and running our MCTS procedure. We then train an Action Chunking with Transformers (ACT) [37] model via behavior cloning on this dataset. During deployment, we roll out the trained policy multiple times in the trial-specific Real2Sim scene, select a rollout that successfully achieves the goal, and execute this trajectory on the real robot. This method succeeded in 5 out of 6 real-world trials, while significantly reducing the average execution time from 644.72 seconds with the MCTS-based search to 72.306 seconds with our policy. Ultimately, these results demonstrate that our MCTS framework can serve as an automated data collection engine to train fast, deployable robot policies.

VI. CONCLUSIONS

We introduce SAIL that reframes robot imitation as an iterative refinement process capable of scaling with test-time compute. By treating full trajectories as objects of search within an MCTS architecture, SAIL enables robots to “think longer” to resolve environmental ambiguities. Experiments across six manipulation tasks demonstrated that increasing the compute budget raised average success rates significantly.

While this work relied on conventional physics simulators for data collection, SAIL is agnostic to the underlying environment. A significant advancement would be to integrate this framework with Digital Twin environments constructed via Gaussian Splatting [38]. By generating datasets that more closely mirror real-world visual statistics, we aim to achieve zero-shot adaptation and bridge the remaining sim-to-real gaps in visual fidelity and contact dynamics.

REFERENCES

- [1] R. Shah, S. Liu, Q. Wang, Z. Jiang, S. Kumar, M. Seo, R. Martín-Martín, and Y. Zhu, “Mimicdroid: In-context learning for humanoid robot manipulation from human play videos,” *arXiv preprint arXiv:2509.09769*, 2025.
- [2] N. Di Palo and E. Johns, “Keypoint action tokens enable in-context imitation learning in robotics,” in *Robotics: Science and Systems (RSS)*, 2024.
- [3] L. Fu, H. Huang, G. Datta, L. Y. Chen, W. C.-H. Panitch, F. Liu, H. Li, and K. Goldberg, “In-context imitation learning via next-token prediction,” *arXiv preprint arXiv:2408.15980*, 2024.
- [4] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid *et al.*, “Rt-2: Vision-language-action models transfer web knowledge to robotic control,” in *Conference on Robot Learning*. PMLR, 2023, pp. 2165–2183.
- [5] G. Puthumanaim, P. Padrao, J. Fuentes, P. Thangeda, W. E. Schafer, L. H. Song, K. Jagdale, L. Bobadilla, and M. Ornik, “Trace: A self-improving framework for robot behavior forecasting with vision-language models,” in *2025 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2025, pp. 11 128–11 135.
- [6] J. Gu, S. Kirmani, P. Wohlhart, Y. Lu, M. G. Arenas, K. Rao, W. Yu, C. Fu, K. Gopalakrishnan, Z. Xu *et al.*, “Rt-trajectory: Robotic task generalization via hindsight trajectory sketches,” *arXiv preprint arXiv:2311.01977*, 2023.
- [7] Q. Lv, H. Li, X. Deng, R. Shao, M. Y. Wang, and L. Nie, “Robomp²: A robotic multimodal perception-planning framework with multimodal large language models,” *arXiv preprint arXiv:2404.04929*, 2024.
- [8] Y. Kuang, J. Ye, H. Geng, J. Mao, C. Deng, L. Guibas, H. Wang, and Y. Wang, “Ram: Retrieval-based affordance transfer for generalizable zero-shot robotic manipulation,” *arXiv preprint arXiv:2407.04689*, 2024.
- [9] Y. Du *et al.*, “Vision-language models as success detectors,” in *Conference on Lifelong Learning Agents (CoLLAs)*, 2023.
- [10] S. A. Sontakke *et al.*, “Roboclip: One demonstration is enough to learn robot policies,” 2023.
- [11] T. Kwon, N. Di Palo, and E. Johns, “Language models as zero-shot trajectory generators,” *IEEE Robotics and Automation Letters*, vol. 9, no. 7, pp. 6728–6735, 2024.
- [12] Y. Yin, Z. Wang, Y. Sharma, D. Niu, T. Darrell, and R. Herzig, “In-context learning enables robot action prediction in llms,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2025.
- [13] W. Xia, D. Wang, X. Pang, Z. Wang, B. Zhao, D. Hu, and X. Li, “Kinematic-aware prompting for generalizable articulated object manipulation with llms,” in *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [14] A. Maurya, T. Ghosh, and R. Prakash, “Trajectory adaptation using large language models,” *arXiv preprint arXiv:2504.12755*, 2025.
- [15] J. Huang *et al.*, “Gelato: Multi-instruction trajectory reshaping via geometry-aware multiagent-based orchestration,” *arXiv preprint arXiv:2509.06031*, 2025.
- [16] A. Curtis, N. Kumar, J. Cao, T. Lozano-Pérez, and L. P. Kaelbling, “Trust the proc3s: Solving long-horizon robotics problems with llms and constraint satisfaction,” in *Conference on Robot Learning (CoRL)*, 2024. [Online]. Available: <https://openreview.net/forum?id=r6ZhiVYriY>
- [17] S. Wang, M. Han, Z. Jiao, Z. Zhang, Y. N. Wu, S.-C. Zhu, and H. Liu, “LLM3: Large language model-based task and motion planning with motion failure reasoning,” *arXiv preprint arXiv:2403.11552*, 2024, IROS 2024. [Online]. Available: <https://arxiv.org/abs/2403.11552>
- [18] Y. Chen, Y. Hao, Y. Zhang, and C. Fan, “Code-as-symbolic-planner: Foundation model-based robot planning via symbolic code generation,” *arXiv preprint arXiv:2503.01700*, 2025. [Online]. Available: <https://arxiv.org/abs/2503.01700>
- [19] H. Chi, Z. Feng, Y. Lyu, C. Zheng, L. Luo, Y.-S. Ong, I. Tsang, H. Chen, Y. Chang, and H. Yin, “Instructflow: Adaptive symbolic constraint-guided code generation for long-horizon planning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. [Online]. Available: <https://openreview.net/forum?id=nzwjvpCO4F>
- [20] K. Sridhar, S. Dutta, D. Jayaraman, and I. Lee, “Regent: A retrieval-augmented generalist agent that can act in-context in new environments,” in *International Conference on Learning Representations (ICLR)*, 2025, oral. [Online]. Available: <https://openreview.net/forum?id=NxyfSW6mLK>
- [21] M. Ahn, A. Brohan *et al.*, “Do as i can, not as i say: Grounding language in robotic affordances,” *arXiv preprint arXiv:2204.01691*, 2022. [Online]. Available: <https://arxiv.org/abs/2204.01691>
- [22] D. Lee, S. Joo, K. Lee, and B. Kim, “Prime the search: Using large language models for guiding geometric task and motion planning by warm-starting tree search,” *arXiv preprint arXiv:2506.07062*, 2025. [Online]. Available: <https://arxiv.org/abs/2506.07062>
- [23] K. Rana *et al.*, “Sayplan: Grounding large language models using 3d scene graphs for scalable robot task planning,” *arXiv preprint arXiv:2307.06135*, 2023. [Online]. Available: <https://arxiv.org/abs/2307.06135>
- [24] Y. J. Ma *et al.*, “Vision language models are in-context value learners,” 2024.
- [25] Q. Chen *et al.*, “Sarm: Stage-aware reward modeling for long horizon robot manipulation,” 2025.
- [26] T. Lee *et al.*, “Roboreward: General-purpose vision-language reward models for robotics,” 2026.
- [27] H. Tan *et al.*, “Robo-dopamine: General process reward modeling for high-precision robotic manipulation,” 2025.
- [28] P. Pacaud *et al.*, “Guardian: Detecting robotic planning and execution errors with vision-language models,” 2025.
- [29] J. Duan *et al.*, “Aha: A vision-language-model for detecting and reasoning over failures in robotic manipulation,” 2024.
- [30] P. Schroeder *et al.*, “Rover: Recursive reasoning over videos with vision-language models for embodied tasks,” 2025.
- [31] C. D. Rosin, “Multi-armed bandits with episode context,” *Annals of Mathematics and Artificial Intelligence*, vol. 61, no. 3, pp. 203–230, 2011.
- [32] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, “The unreasonable effectiveness of deep features as a perceptual metric,” in *CVPR*, 2018.
- [33] Gemini Robotics Team *et al.*, “Gemini Robotics: Bringing AI into the physical world,” *arXiv preprint arXiv:2503.20020*, 2025. [Online]. Available: <https://arxiv.org/abs/2503.20020>
- [34] R. Cadene, S. Alibert, F. Capuano, M. Aractingi, A. Zouitine, P. Kooijmans, J. Choghari, M. Russi, C. Pascal, S. Palma, M. Shukor, J. Moss, A. Soare, D. Aubakirova, Q. Lhoest, Q. Gallouédec, and T. Wolf, “Lerobot: An open-source library for end-to-end robot learning,” in *International Conference on Learning Representations (ICLR)*, 2026, poster. [Online]. Available: <https://openreview.net/forum?id=CiZMMAFQR3>
- [35] S. Liu, Z. Zeng, T. Ren, F. Li, H. Zhang, J. Yang, Q. Jiang, C. Li, J. Yang, H. Su, J. Zhu, and L. Zhang, “Grounding DINO: Marrying DINO with grounded pre-training for open-set object detection,” *arXiv preprint arXiv:2303.05499*, 2023. [Online]. Available: <https://arxiv.org/abs/2303.05499>
- [36] N. Ravi, V. Gabeur, Y.-T. Hu, R. Hu, C. Ryali, T. Ma, H. Khedr, R. Rädle, C. Rolland, L. Gustafson, E. Mintun, J. Pan, K. V. Alwala, N. Carion, C.-Y. Wu, R. Girshick, P. Dollár, and C. Feichtenhofer, “SAM 2: Segment anything in images and videos,” *arXiv preprint arXiv:2408.00714*, 2024. [Online]. Available: <https://arxiv.org/abs/2408.00714>
- [37] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, “Learning fine-grained bimanual manipulation with low-cost hardware,” *arXiv preprint arXiv:2304.13705*, 2023. [Online]. Available: <https://arxiv.org/abs/2304.13705>
- [38] G. Jiang, H. Chang, R.-Z. Qiu, Y. Liang, M. Ji, J. Zhu, Z. Dong, X. Zou, and X. Wang, “GSWorld: Closed-loop photo-realistic simulation suite for robotic manipulation,” *arXiv preprint arXiv:2510.20813*, 2025. [Online]. Available: <https://arxiv.org/abs/2510.20813>